

Pipeline de Captura y Publicación de Datasets para Entrenamiento de LLMs

GIDEAL — Workspace ELECTRONICA

Prof. César Rodríguez

6 de agosto de 2026

Resumen ejecutivo

Se desarrolló e integró una **cadena completa de valor de datos** dentro del workspace ELECTRONICA: la captura pasiva de muestras de entrenamiento generadas por los flujos existentes (RAG, análisis de imágenes y conversión imagen a KiCad), su curado y particionado, su empaquetado en formato Hugging Face con metadata de licencia, y su publicación en el Hugging Face Hub. Todo siguiendo la arquitectura de tres capas del proyecto: directivas, orquestación y ejecución.

1. Arquitectura de 3 capas

Capa	Dónde	Rol
Directivas	<code>directives/*.yaml</code>	SOP: qué hacer
Orquestación	<code>flujo_*.py</code> (raíz)	Decisiones y validación
Ejecución	<code>execution/*.py</code>	Scripts deterministas: cómo

La lógica probabilística (LLMs) se mantiene en los orquestadores; todo el procesamiento de datos vive en scripts deterministas y reutilizables.

2. Cadena de valor: 4 eslabones

1. **Captura pasiva** — registra muestras sin interrumpir los flujos
2. **Curado** — filtra, valida, deduplica y particiona
3. **Empaquetado** — genera paquetes Hugging Face con licencia
4. **Publicación** — sube al Hugging Face Hub

2.1. 1. Captura pasiva

Tres orquestadores existentes fueron instrumentados para capturar sus salidas en formato JSONL dentro de `datasets/` (excluido de git):

- `rag_system.py` → `datasets/rag_conversaciones.jsonl` (ShareGPT)
- `flujo_imagen_a_kicad.py` → `datasets/eda_imagen_circuito.jsonl`
- `flujo_analizar_imagen.py` → `datasets/analisis_imagenes.jsonl`

La captura es 100% pasiva: toda llamada va en `try/except`, sanitiza PII (emails, cédulas, teléfonos → [REDACTADO]), descarta duplicados por hash canónico y respeta la variable `DATA_CAPTURE_DISABLED` o el flag `-no-capture-data`.

2.2. 2. Curado

`execution/curar_datasets.py` lee los JSONL crudos y produce `datasets/curated/`: valida el esquema, descarta registros con `quality_score` bajo el umbral (default 0.7), deduplica y particiona en `train/val/test` (default 80-10-10, semilla fija 42). Emite `manifest.json` y `resumen.md`.

2.3. 3. Empaquetado

`execution/empaquetar_dataset.py` convierte los datasets curados en paquetes formato Hugging Face en `datasets/paquetes/`: `dataset_info.json`, `README.md` (tarjeta con metadata), `LICENSE` y splits en Parquet o JSONL. Soporta licencias `custom-proprietary` (GIDEAL-1.0), `apache-2.0`, `cc-by-4.0` y `mit`, además de empaquetado en `.tar.gz` con el flag `-pack`.

2.4. 4. Publicación en Hugging Face Hub

`execution/publicar_hf.py` lee `HF_TOKEN` del entorno o de `.env`, crea el repositorio de datasets (con `exist_ok=True`) y sube el contenido del paquete. Retorna la URL pública del repositorio. El flag `-private` genera repositorios privados.

3. Esquema de los datasets

- **RAG (ShareGPT):** `messages[]` (`system/user/assistant`) + `metadata.retrieved_context[]`, `domain`, `model`, `quality_score`.
- **EDA imagen→circuito:** `images[]`, `instruction`, `output` (`components[]` y `connections[]`) + `api_backend`, `model`, `tokens_usados`.
- **Análisis multimodal:** `images[]`, `instruction`, `output` (descripción, elementos detectados) + metadata de modelo.

Todo registro incluye `id`, `schema_version` y `timestamp`.

4. Comandos de uso

```
# Curado
python3 flujo_curar_dataset.py [--min-quality 0.7] [--split 80-10-10]
    [--dry-run] [--no-alert]

# Empaquetado
python3 flujo_empaquetar_dataset.py [--format parquet|jsonl] [--license
    <lic>] [--pack] [--no-alert]

# Publicación
python3 flujo_publicar_hf.py <dataset> [--repo <id>] [--private]
    [--dry-run] [--no-alert]
```

5. Archivos creados y modificados

- **Creados:** `execution/data_capture.py`, `execution/curar_datasets.py`, `execution/empaquetar_dataset.py`, `execution/publicar_hf.py`, `flujo_curar_dataset.py`, `flujo_empaquetar_dataset.py`, `flujo_publicar_hf.py` y tres directivas (`directives/data_capture.yaml`, `curar_dataset.yaml`, `empaquetar_dataset.yaml`, `publicar_hf.yaml`).
- **Modificados:** `rag_system.py`, `flujo_imagen_a_kicad.py`, `flujo_analizar_imagen.py`, `.gitignore` (se excluye `datasets/`) y `AGENTS.md`.

6. Prueba real publicada

Se publicó el dataset de ejemplo `rag_conversaciones` en <https://huggingface.co/datasets/cero2k6/gideal-rag-v1> con el token recuperado de la caché de Hugging Face del usuario `cero2k6`. El repositorio contiene `LICENSE`, `README.md`, `dataset_info.json` y `train.parquet`, y queda como referencia de estructura. Se corrigió durante el desarrollo que Hugging Face exige IDs de licencia válidos en el frontmatter del `README`: las licencias propias se registran como `other + license_name/license_link`.

7. Consideraciones y convenciones

- El directorio local `datasets/` **sombrea la librería** `datasets` de Hugging Face desde la raíz del workspace; los Parquet se cargan con `pyarrow` (ver `README` de los paquetes).
- `datasets/` y `.env` están excluidos de git (`.gitignore`).
- El token HF jamás se imprime ni registra; se lee de `.env` o del entorno.
- La licencia GIDEAL-1.0 no es de uso público libre: para datos comerciales se recomienda publicar con `-private`.
- Con menos de 10 muestras, todo el split va a train (val/test en 0); al acumular más datos, el particionado se reparte automáticamente.

8. Trabajo futuro

- Acumular datos reales y publicar datasets con cientos de muestras.
- Automatizar el curado periódico (cron).
- Integrar la cadena con la estrategia Web3 de comercialización de datos (licencias on-chain, verificación).
- Añadir validación de calidad por heurísticas (longitud, consistencia del netlist, cobertura de imágenes).